南開大學
Nankai University

计算机学院

并行程序设计实验报告

Lab1：体系结构相关编程

——Cache 与超标量优化的程序性能测试

姓名：
学号：
专业：计算机科学与技术
班级：
源码链接：

2026年 4月 1日

1 实验目的与基本要求

本实验围绕现代处理器中的两类关键性能因素展开：一类是由存储层次结构决定的 cache 局部性，另一类是由流水线与发射宽度决定的指令级并行能力。根据课程 PPT 与 Lab1 实验说明，本次基础部分需要完成两项任务：其一，计算给定 $n \times n$ 矩阵每一列与给定向量的内积，分别实现逐列访问的平凡算法和面向 cache 的优化算法；其二，计算 n 个数之和，分别实现逐个链式累加的平凡算法和适合超标量处理器的优化算法。在此基础上，程序还需采用高精度计时方法，对不同问题规模进行多次测试，并在实验报告中说明程序设计思路、实验平台、测试方案、实验结果及分析。

本次实验在本地 x86 平台完成。实际运行前先进行了正确性检查，终端输出为 `Correctness check passed.`，说明各版本程序在固定测试数据下结果一致。后续计时采用重复执行取平均值的方式，以减小单次运行时间过短带来的测量误差。

2 实验平台与测试方案

实验平台为 Intel Core i9-14900HX 处理器，基准频率 2.20 GHz，24 个物理核心，32 个逻辑处理器，内存容量 31.8 GB，操作系统为 Microsoft Windows 11 家庭中文版 64 位，编译器为 TDM-GCC 10.3.0。根据任务管理器与脚本采集结果，处理器的 L1 Cache 为 2.1 MB，L2 Cache 为 32.0 MB，L3 Cache 为 36.0 MB。程序在 VS Code 终端环境下编译和运行，基础测试采用 `-O2` 编译选项。

测试数据均采用固定规则生成，以保证结果可复现并便于进行正确性校验。矩阵内积实验选取 $n = 64, 128, 256, 384, 512, 768, 1024, 15$ 九组规模；求和实验选取 $n = 1024$ 至 16777216 的八组规模。考虑到小规模问题单次运行时间过短，测试程序会根据规模自动调整重复次数，并记录平均运行时间。输出结果保存为 CSV 文件，其中包含每组问题规模的平均时间、加速比以及重复执行次数。

3 实验一：矩阵每列与向量内积

3.1 算法设计

设矩阵为 $B \in \mathbb{R}^{n\times n}$，向量为 $a \in \mathbb{R}^n$，结果向量为 $sum \in \mathbb{R}^n$，其中第 i 个结果满足

$$sum_i = \sum_{j=0}^{n-1} B_{j,i} \cdot a_j.$$

平凡算法直接按照“按列求内积”的思路实现：外层循环固定列号 i，内层循环遍历所有行号 j，逐项累加 $B_{j,i} \cdot a_j$。该算法逻辑最直观，但在 C 语言的行主存储下，对矩阵元素的访问步长为 n，会造成跨行跳跃访问，空间局部性较差。

cache 优化算法并不改变数学计算量，而是调整循环顺序：先遍历矩阵的每一行，再把该行对所有列结果的贡献一次性累加到结果向量中。这样内层循环访问的是一整行连续存储的数据，能够更充分地利用 cache line 中的相邻元素，减少无效访存和缓存失效带来的性能损失。

3.2 实验结果

表 1 给出了在 `-O2` 编译下两种算法的基础测试结果。

表 1: 矩阵每列与向量内积基础测试结果

n	naive_ms	cache_ms	speedup	repeats_naive	repeats_cache
64	0.000903	0.001141	0.791419	292960	292960
128	0.006789	0.004073	1.667030	36616	73232
256	0.060795	0.031533	1.927994	4576	9152
384	0.129354	0.079284	1.631521	2032	4064
512	0.254104	0.115108	2.207516	1144	2288
768	0.700431	0.277682	2.522421	508	1016
1024	3.335070	0.500330	6.665738	71	568
1536	7.004510	1.146008	6.112097	31	248
2048	37.766971	3.649994	10.347132	17	68

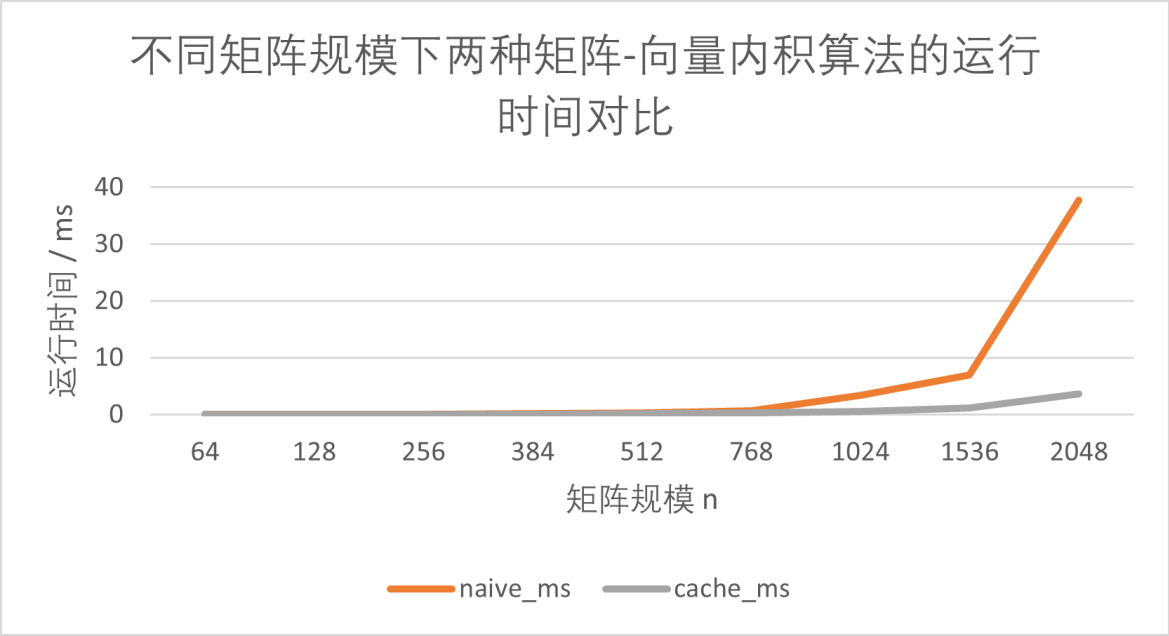

图 1: 不同矩阵规模下平凡算法与 cache 优化算法的运行时间对比

3.3 结果分析

从表 1 可以看出，在问题规模较小时，两种算法的时间差距不大；例如 $n = 64$ 时，cache 优化版本甚至略慢于平凡算法。这说明当数据规模较小时，矩阵数据仍能较好地驻留在低层缓存中，单纯调整循环顺序所带来的收益尚不明显。

随着 n 增大，cache 优化算法的优势迅速体现出来。$n = 512$ 时加速比已经达到 2.21 倍；当 $n = 1024$ 时加速比提升到 6.67 倍；到 $n = 2048$ 时进一步达到 10.35 倍。这表明该实验的性能瓶颈主要不是乘加次数，而是访存模式。平凡算法按列访问矩阵，在行主存储布局下会频繁跨越不同 cache line，导致缓存利用率较低；而优化算法按行连续读取矩阵元素，能够显著改善空间局部性，因此在规模增大后表现出明显更高的吞吐能力。

结合本机 2.1 MB 的 L1 Cache、32.0 MB 的 L2 Cache 和 36.0 MB 的 L3 Cache 可以进一步理解这一现象：随着矩阵规模增大，数据逐渐超过低层缓存容量，访存模式对性能的影响被放大，因此两种算法之间的差距在大规模区间显著拉大。

4 实验二：n 个数求和

4.1 算法设计

第二个实验要求计算长度为 n 的数组元素之和。平凡算法使用单个累加器，从头到尾逐个执行 `sum += a[i]`。这种写法虽然最简单，但每一步运算都依赖上一步的结果，会形成较长的数据相关链，从而限制处理器流水线和超标量发射单元的并行能力。

优化算法采用两路链式累加的思路：使用两个独立的累加器分别处理不同位置的数组元素，最后再把两个部分和相加。这样可以将一条很长的依赖链拆分为两条较短且彼此独立的依赖链，从而更好地发挥处理器的指令级并行能力。该优化并未减少总加法次数，但改善了指令调度条件，因此有望提高执行效率。

4.2 实验结果

表 2 给出了在 `-O2` 编译下链式平凡算法和两路链式累加算法的测试结果。

表 2: n 个数求和基础测试结果

n	naive_ms	dual_ms	speedup	repeats_naive	repeats_dual
1024	0.000875	0.000533	1.639727	390624	390624
4096	0.004069	0.002179	1.867015	97656	195312
16384	0.016338	0.007393	2.209832	24414	48828
65536	0.059309	0.032988	1.797910	6102	6102
262144	0.235940	0.125555	1.879175	1524	3048
1048576	1.124038	0.594254	1.891511	190	380
4194304	4.968629	3.002182	1.655006	94	94
16777216	19.795482	19.021014	1.040716	11	22

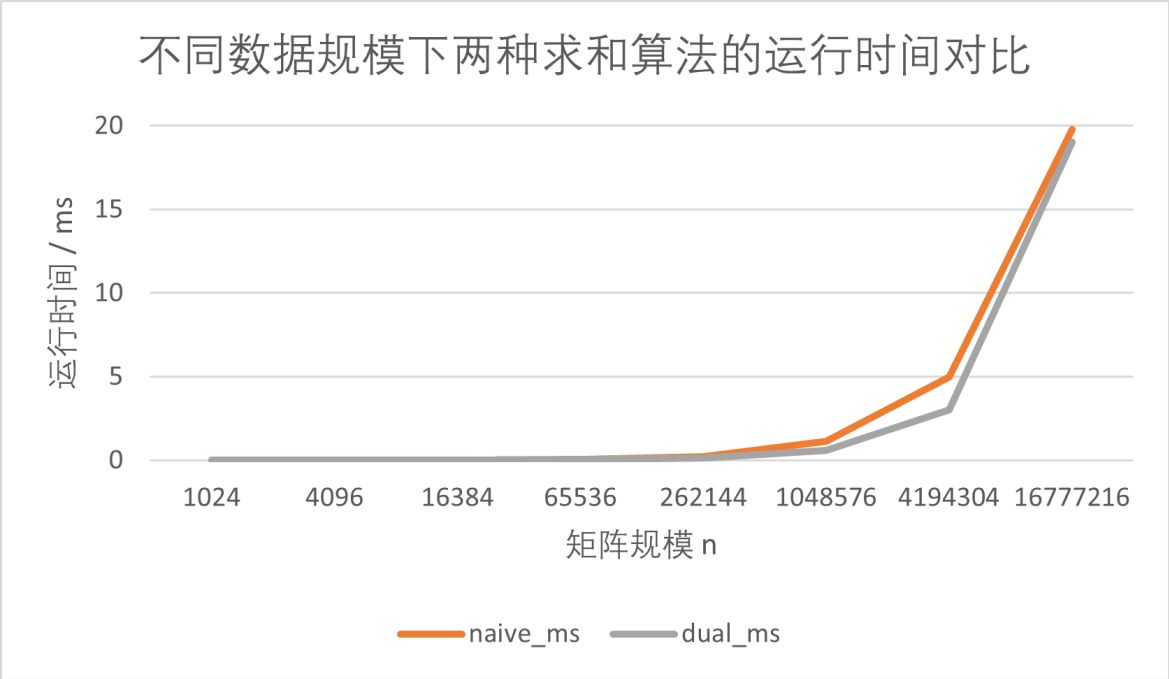

图 2: 不同数据规模下链式累加与两路链式累加的运行时间对比

4.3 结果分析

从表 2 可以看出，两路链式累加算法在大多数规模下都优于平凡算法。较为典型的是 $n = 16384$ 时，加速比达到 2.21 倍；在 $n = 262144$ 和 $n = 1048576$ 时，加速比分别为 1.88 倍和 1.89 倍。说明把单累加器改为双累加器之后，确实能够减少指令之间的串行依赖，提高处理器的并行执行效率。

不过当规模继续增大到 $n = 16777216$ 时，加速比下降到 1.04 倍，优化效果明显减弱。这说明在超大规模下，程序性能已不再主要受算术依赖限制，而更多受到内存访问带宽、缓存层次和总体数据搬移开销的影响。换言之，在问题规模较小时，超标量优化能够明显提升求和效率；而当数据规模巨大时，访存开销逐渐成为新的主要瓶颈，导致双链路累加相对平凡算法的优势被削弱。

5 结论

本次实验围绕 cache 与超标量两类典型体系结构特性，对两个简单但具有代表性的计算问题进行了实现与测试。对于矩阵每列与向量内积问题，平凡算法和 cache 优化算法的主要差异体现在访存顺序上。实验结果表明，随着矩阵规模增大，按行连续访问的优化版本能够显著降低缓存失效带来的性能损失，在大规模下取得超过 10 倍的加速效果。对于 n 个数求和问题，两路链式累加通过缩短单条依赖链、提高指令级并行性，在多数规模下都优于逐个累加的平凡算法，但其优势会在超大规模数据下因访存带宽限制而减弱。

总体来看，本次实验验证了两个结论：第一，程序性能不仅取决于算法的理论计算量，还与数据在存储层次中的组织方式密切相关；第二，合理减少指令之间的数据相关性，可以使处理器更充分地发挥超标量执行能力。这些现象都说明，面向现代体系结构进行程序优化时，必须同时关注计算过程本身和底层硬件的数据流特征。